

OFFSHORE MANAGEMENT SYSTEM

Technical Specification Document

Version 1.0

Built with .NET Core & Angular
Clean Architecture Pattern

5/6/2026

EXECUTIVE SUMMARY

The Offshore Management System is a comprehensive enterprise solution designed to streamline project management, team coordination, and client service delivery. This document provides technical specifications for implementation using modern technologies and best practices.

Key Objectives:

- Centralized platform for managing offshore services and client projects
- Real-time project tracking and team collaboration
- Seamless integration between admin, client, and public systems
- Secure authentication and authorization with JWT
- Scalable architecture supporting future enhancements

Technology Stack:

- Backend: ASP.NET Core 8+ with C#
- Frontend: Angular 17+
- Database: SQL Server 2019+
- Architecture: Clean Architecture
- Authentication: JWT (JSON Web Tokens)

TABLE OF CONTENTS

1. System Overview
2. Architecture Design
3. Backend Structure
4. Frontend Architecture
5. Database Design

- 6. Authentication & Security
- 7. Core Modules & Features
- 8. Implementation Tasks
- 9. Development Roadmap

1. SYSTEM OVERVIEW

1.1 Purpose

The Offshore Management System provides an integrated platform for managing outsourced services, project delivery, and team collaboration. It enables admins to manage resources and services while allowing clients to request services and monitor progress.

1.2 System Components

- Admin Panel: Centralized management of teams, services, clients, and projects
- Client Portal: Self-service platform for requesting and tracking services
- Public Website: Marketing platform with service browsing and contact forms

1.3 User Roles

System Administrator (Admin):

- Full system access and control
- Manage team members, skills, and assignments
- Monitor all projects and requests

Registered Client:

- Create and submit service requests
- Real-time project status monitoring
- View team capabilities and portfolio

Website Visitor:

- Browse services
- Submit service requests via forms

2. ARCHITECTURE DESIGN

2.1 Clean Architecture Overview

The system follows Clean Architecture principles to ensure maintainability, testability, and scalability. The architecture is organized in concentric layers with specific responsibilities.

2.2 Architecture Layers

Presentation Layer:

- Handles HTTP requests, responses, and user interactions

Application Layer:

- Contains business logic, use cases, services, and validators

Domain Layer:

- Defines core entities, value objects, and business rules

Infrastructure Layer:

- Data access, repositories, external services, and implementations

3. BACKEND STRUCTURE (.NET CORE)

3.1 Project Organization

Layer/Project	Purpose	Key Components
Domain	Core business logic	Entities, Value Objects
Application	Use cases & services	Services, DTOs, Validators
Infrastructure	Data access	Repositories, DbContext
API	HTTP endpoints	Controllers, Middleware

3.2 Core Components

Controllers:

- AuthController - Registration, login, token refresh
- TeamController - CRUD for team members
- ServiceController - Service management
- RequestController - Service request handling
- ProjectController - Project tracking

Application Services:

- AuthService - JWT token generation and validation
- TeamService - Team member management
- RequestService - Request lifecycle management
- ProjectService - Project operations
- EmailService - Notification sending

Domain Entities:

- User - System users with roles
- TeamMember - Employee data
- Service - Service offerings
- ServiceRequest - Client requests
- Project - Project tracking

4. FRONTEND ARCHITECTURE (ANGULAR)

4.1 Project Structure

Folder	Purpose	Contents
core	Shared services	Auth service, HTTP client
modules	Feature modules	Auth, Admin, Client, Public
shared	Common components	Headers, Footers, Navbars
assets	Static resources	Styles, Icons, Images

4.2 Key Features

- Lazy-loaded modules for performance
- HTTP interceptors for JWT token handling
- Route guards for authentication
- RxJS for reactive data flows

5. DATABASE DESIGN (SQL SERVER)

5.1 Key Tables

Table	Purpose	Key Fields
Users	System users	UserId, Email, Role
TeamMembers	Employee data	Skills, Experience, CV
Clients	Client profiles	Company, Contact
Services	Service offerings	Category, Name
ServiceRequests	Request tracking	Status, ClientId, Service

5.2 Database Relationships

- User (1) → Many (TeamMember, Client)
- TeamMember (1) → Many (Project assignments)
- Client (1) → Many (ServiceRequest)
- ServiceRequest (1) → Many (ServiceRequestTeam)
- Service (1) → Many (ServiceRequest, Portfolio)

5.3 Database Specifications

- Database: SQL Server 2019+
- Collation: SQL_Latin1_General_CP1_CI_AS
- Approach: Entity Framework Core Code-First
- Backup: Daily automated backups

6. AUTHENTICATION & SECURITY

6.1 JWT Authentication

The system uses JWT for stateless authentication with the following specifications:

- Header: HS256 algorithm
- Payload: UserId, Email, Role, Permissions
- Access Token: 15-minute expiration

6.2 Password Security

- Algorithm: bcrypt with salt
- Minimum: 8 chars, mixed case, numbers, special characters
- Hashed before database storage

6.3 Authorization

- Role-Based Access Control (RBAC)
- Endpoint protection via [Authorize] attributes
- Custom policies for complex scenarios

6.4 Security Best Practices

- HTTPS only for all communications
- CORS configuration for frontend domain
- Input validation and sanitization
- SQL injection prevention with parameterized queries
- XSS protection with Content Security Policy
- Rate limiting on authentication endpoints

7. CORE MODULES & FEATURES

7.1 Authentication Module

Features:

- User registration with email verification
- Login with JWT token generation
- Token refresh mechanism
- Password reset functionality

7.2 Team Management Module

Features:

- Add/edit/delete team members
- Skills and experience tracking
- Team structure with leaders
- CV generation and storage

7.3 Service Management Module

Features:

- Create/manage service categories
- Add/edit/delete services
- Service visibility control

7.4 Service Request Module

Features:

- Create requests from website/client portal
- Status tracking (Pending, In Progress, Completed)
- Automatic email notifications

7.5 Project Management Module

Features:

- Project creation from requests
- Team member assignment
- Status tracking and monitoring

7.6 Client Management Module

Features:

- Client registration and profiles
- View team capabilities
- Browse portfolio

7.7 Portfolio Module

Features:

- Add/manage portfolio projects
- Associate with services
- Upload project images

8. IMPLEMENTATION TASKS

8.1 Phase 1: Infrastructure (Week 1-2)

1. T1.1: Create .NET Core project structure
2. T1.2: Configure Entity Framework and database
3. T1.3: Create database migrations
4. T1.4: Set up Angular project structure
5. T1.5: Configure HTTP client and interceptors

8.2 Phase 2: Authentication (Week 3-4)

6. T2.1: Implement Identity Framework
7. T2.2: Create JWT token service
8. T2.3: Build authentication controllers
9. T2.4: Configure CORS and security
10. T2.5: Create auth guards and login components

8.3 Phase 3: Core Modules (Week 5-8)

- 11. T3.1: Implement team management
- 12. T3.2: Implement service management
- 13. T3.3: Implement request management
- 14. T3.4: Implement project management
- 15. T3.5: Implement client management
- 16. T3.6: Implement portfolio management

8.4 Phase 4: Integration & Testing (Week 9-10)

- 17. T4.1: Integration testing
- 18. T4.2: End-to-end testing
- 19. T4.3: Performance optimization
- 20. T4.4: Security audit

8.5 Phase 5: Deployment (Week 11)

- 21. T5.1: Database deployment
- 22. T5.2: Backend deployment
- 23. T5.3: Frontend deployment
- 24. T5.4: Go-live and monitoring

9. DEVELOPMENT ROADMAP

Phase	Timeline	Focus	Deliverables
1	Week 1-2	Infrastructure	DB, Project setup
2	Week 3-4	Authentication	JWT, Login/Register
3	Week 5-8	Core Modules	Team, Services, Requests
4	Week 9-10	Testing	Integration, QA
5	Week 11	Deployment	Go-live, Monitoring

CONCLUSION

The Offshore Management System is designed as a robust, scalable, and secure enterprise solution using modern technologies and Clean Architecture principles. This technical specification provides a comprehensive roadmap for development with clear task definitions, organized module structure, and a realistic 11-week implementation timeline.

The modular design ensures components can be developed in parallel, JWT authentication provides secure access control, and SQL Server database ensures data integrity. This combination creates a professional platform meeting all offshore service management requirements.

This document serves as the foundation for development and should be referenced throughout the implementation process.